

Calcul Parallèle



**TD1: PROGRAMMATION PYTHON
MPDS₁**

DR. SAMYA AMIRI

Faire des calculs avec Python



Exercice 1 : Quelques exemples de calcul

Essayez, en les exécutant, de comprendre ce que fait chaque instruction (non commentée) de l'exemple ci-dessous. Cet exemple est valable en ligne de commande uniquement.

En ligne de commande, la valeur d'une variable ou le résultat d'un calcul s'affiche directement après la saisie de cette variable ou de ce calcul.

```
>>> 5+3
8
>>>5*3
15
>>>5**3
125
>>> x=1 # declaration d'un variable x de valeur 1 (# pour le commentaire)
>>> x    # affichage de x
1
>>> a,b,c=3,5,7 # declaration de 3 variables a, b et c de valeurs resp. 3, 5 et 7
>>> a-b/c
2.2857142857142856
>>> (a-b)/c
-0.2857142857142857
```

Faire des calculs avec Python



```
>>> b/c  
0.7142857142857143
```

```
>>> b//c  
0
```

```
>>> b%c  
5
```

```
>>> d=1.1
```

```
>>> d/c  
0.15714285714285717
```

```
>>> d//c  
0.0
```

```
>>> from math import * # Pour importer la librairie de fonctions mathematiques
```

```
>>> sqrt(4) # Pour calculer la racine carree
```

```
2.0
```

```
>>> pi
```

```
3.141592653589793
```

Affichage



Exemple 2 : Utilisation de la fonction d'affichage `print()`

```
>>> print(a+b) # a et b sont les variable de l'exercice 1
8
>>> print('la valeur de', a, '+', b, 'est :', a+b)
('la valeur de', 3, '+', 5, 'est :', 8)
```

Déclaration et initialisation de variables et types



Exemple 3

```
>>> print(type(a)) # a est la variable de l'exercice 1
<class 'int'>
>>> pi=3,14
>>> print(type(pi))
<class 'tuple'>
>>> pi=3.14
>>> print(type(pi))
<class 'float'>
>>> s='exemple de chaine de caracteres'
>>> type(s)
<class 'str'>
>>> 2+'1.5'
Traceback (most recent call last):
  File "<console>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
>>> 2+eval('1.5') # Pour \ 'eliminer l'erreur pr\ 'ec\ 'edente
3.5
```

Chaînes de caractères



Exemple 4 : Manipulation des chaîne de caractères et exemples de fonctions sur les chaînes de caractère

```
>>> s='un exemple de chaine'
>>> s2="un autre exemple"
>>> s[1] # Acces au caractere d'indice 1 (les indices commencent a zero)
'n'
>>> print(s[0],s2[0])
u u
>>> print(s[4],s2[0])
x u
>>> print(s + ' et ' + s2) # Concatenation de chaines
un exemple de chaine et un autre exemple
>>> s3=s + ' et ' + s2
>>> s3
'un exemple de chaine et un autre exemple'
>>> s2*2
'un autre exempleun autre exemple'
>>> print('La taille de s est :', len(s))
La taille de s est : 20
>>> s3[0:3] # Recuperation des caracteres de position entre les 0 et 3e
'un '
```

Chaînes de caractères



```
>>> s3[4:8]
'xemp'
>>> print(s3[:3]) # Recuperation des 3 premiers caracteres
un
>>> print(s3[3:]) # Recuperation des caracteres a partir de la position 3
exemple de chaine et un autre exemple
>>> s3[::-1]
'elpmexe ertua nu te eniahc ed elpmexe nu'
>>> s3.find("exemple")
3
>>> s3.replace("chaine", "str")
'un exemple de str et un autre exemple'
>>> help(str) # pour afficher l'aide sur la classe str
```

Exemple 5 : Exemple de récupération des mots d'une chaîne de caractères

```
>>> sentence = 'It is raining cats and dogs'
>>> words = sentence.split()
>>> print(words)
['It', 'is', 'raining', 'cats', 'and', 'dogs']
```

Boucles et conditions



```
>>> s3[4:8]
'xemp'
>>> print(s3[:3]) # Recuperation des 3 premiers caracteres
un
>>> print(s3[3:]) # Recuperation des caracteres a partir de la position 3
exemple de chaine et un autre exemple
>>> s3[::-1]
'elpmexe ertua nu te eniahc ed elpmexe nu'
>>> s3.find("exemple")
3
>>> s3.replace("chaine", "str")
'un exemple de str et un autre exemple'
>>> help(str) # pour afficher l'aide sur la classe str
```

Exemple 5 : Exemple de récupération des mots d'une chaîne de caractères

```
>>> sentence = 'It is raining cats and dogs'
>>> words = sentence.split()
>>> print(words)
['It', 'is', 'raining', 'cats', 'and', 'dogs']
```

Boucles et conditions



A partir d'ici, vous pouvez commencer à saisir les instructions dans un fichier d'extension .py et vous pouvez exécuter ce fichier.

Attention : En Python il n'y a pas, comme dans certains langages, d'accolade ouvrante ou fermante pour délimiter un bloc d'instructions. Les blocs d'instructions en python sont délimités par ":" puis des tabulations : toutes les instructions consécutives à un ":" et débutant par un même nombre de tabulations appartiennent à un même bloc d'instructions.

Boucles et conditions

10

Exercice 2 : Boucle for

Tapez le code suivant et observez le résultat.

```
for i in range(10): # Ne pas oublier les deux points!!
    x = 2 # Attention ne pas oublier une tab. en debut de ligne sinon erreur!!!
    print(x*i) # Ne pas oublier la tabulation en debut de ligne!!
# Tapez encore une fois <Enter> si vous Ãªtes en ligne de commande
```

Exercice 3 : Boucle while

Tapez le code suivant et observez le résultat.

```
a=0
while(a<12): # Ne pas oublier les deux points!!
    a=a+1 # Ne pas oublier la tabulation en debut de ligne!!
    print(a, a**2,a**3) # Ne pas oublier la tabulation en debut de ligne!!
# Tapez encore une fois <Enter> si vous Ãªtes en ligne de commande
```

Boucles et conditions

11

Exercice 4 : Condition If/Then/Else

Tapez le code suivant et observez le résultat.

```
a=0
if a==0: # Ne pas oublier les deux points!!
    print('0') # Ne pas oublier la tabulation en debut de ligne!!
elif a==1: # Ne pas mettre de tabulation et ne pas oublier les deux points!!
    print('1') # Ne pas oublier la tabulation en debut de ligne!!
else: # Ne pas mettre de tabulation et ne pas oublier les deux points!!
    print('2') # Ne pas oublier la tabulation en debut de ligne!!
# Tapez encore une fois <Enter> si vous Ãªtes en ligne de commande
```

Récupérer des saisies claviers

12

Exemple 6

```
>>> s=input() # taper <Enter> puis saisir quelque chose au clavier
>>> s
>>> s=input("Saisir une chaine :") # taper <Enter>
Saisir une chaine :
>>> s
```

Exercice 5 : *Ecrivez un script permettant d'obtenir le résultat suivant*

```
Saisissez une chaine
une chaine
La chaine inversee est:
eniahc enu
```

Récupérer des saisies claviers

13

Exemple 6

```
>>> s=input() # taper <Enter> puis saisir quelque chose au clavier
>>> s
>>> s=input("Saisir une chaine :") # taper <Enter>
Saisir une chaine :
>>> s
```

Exercice 5 : *Ecrivez un script permettant d'obtenir le résultat suivant*

```
Saisissez une chaine
une chaine
La chaine inversee est:
eniahc enu
```

Structures de données : Listes

14

Exercice 7 : Manipulation des listes

Tapez chacune des instructions suivantes (en ligne de commande) et observez le résultat.

```
>>> list=['lundi', 2, 'janvier']
>>> print(list)
>>> list[0] # Mettre print(list[0]) si vous n'êtes pas en ligne de commandes
>>> list[-1]
>>> print(list[2])
>>> len(list)
>>> list.append(2010)
>>> list
>>> list[3]=list[3]+1
>>> list[3]
>>> del list[0]
>>> list
>>> list.insert(0,'mardi')
```

Structures de données : Listes

15

```
>>> list
>>> 'mardi' in list
>>> 'lundi' in list
>>> list.index("mardi")
>>> list2=list[1:3]
>>> list2
>>> list3=list[:2]
>>> list3
>>> list4=list[1:]
>>> list4
>>> list5=list[-3:-1]
>>> list5
>>> list6 = list[: -1]
>>> list6
>>> list3=list3 + [2011]
```

Structures de données : Listes

16

```
>>> list3
>>> list7=3*list
>>> list7
>>> list.extend([3,4])
>>> list
>>> list=list.pop(0)
>>> list
>>> list=[1,2,3]
>>> list2=list # Attention list et list2 correspondent a la meme liste!!
>>> list
>>> list.pop(1)
>>> list
>>> list2
>>> list=[1,2,3]
>>> list2=list.copy() # list2 est une copie de list
>>> list.pop(1)
>>> list
>>> list2
```

Structures de données : Listes

17

```
>>> list=[1,"ab",[1,True]] # liste imbriquee
>>> list[2]
>>> print(list(range(10)))
>>> print(list(range(1,10)))
>>> print(list(range(1,10,3)))
>>> help(list) # pour l'aide sur les listes
```

Exemple 7 : Saisies d'une liste au clavier

```
>>> list = [input(),input(),input()]
>>> list
>>> list=[x for x in input("Saisir les elements de la liste separees
par une virgule (ex. 1,2,"abc") :").split(',')]] # Taper <Enter>
>>> list
>>> list=[int(x) for x in input("Saisir des entiers separees
par une virgule (ex. 1,2,"abc") :").split(',')]] # Taper <Enter>
>>> list
```

Structures de données : Listes

18

```
>>> list=[1,"ab",[1,True]] # liste imbriquee
>>> list[2]
>>> print(list(range(10)))
>>> print(list(range(1,10)))
>>> print(list(range(1,10,3)))
>>> help(list) # pour l'aide sur les listes
```

Exemple 7 : Saisies d'une liste au clavier

```
>>> list = [input(),input(),input()]
>>> list
>>> list=[x for x in input("Saisir les elements de la liste separees
par une virgule (ex. 1,2,"abc") :").split(',')]] # Taper <Enter>
>>> list
>>> list=[int(x) for x in input("Saisir des entiers separees
par une virgule (ex. 1,2,"abc") :").split(',')]] # Taper <Enter>
>>> list
```

Structures de données : Listes

19

Exercice 8 : *Ecrivez un programme qui demande à l'utilisateur d'entrer des notes d'élèves. Si l'utilisateur entre une valeur négative, le programme s'arrête. En revanche, pour chaque note saisie, le programme construit progressivement une liste. Après chaque entrée d'une nouvelle note (et donc à chaque itération de la boucle), il affiche le nombre de notes entrées, la note la plus élevée, la note la plus basse, la moyenne de toutes les notes.*

Astuce : *Pour créer une liste vide, il suffit de taper la commande `list = []`.*

Structures de données : Dictionnaires

20

Exercice 9 : Tapez le code suivant² et observez le résultat.

```
>>> dico = {} # dictionnaire vide
>>> dico['computer'] = 'ordinateur'
>>> dico['mouse'] = 'souris'
>>> dico['keyboard'] = 'clavier'
>>> print(dico)
>>> print(dico.keys())
>>> print(dico.values())
>>> del dico['mouse']
>>> print(dico)
>>> print(len(dico))
>>> dico.__contains__('computer')
>>> print(dico.items())
>>> for clef in dico: # Ne pas oublier les 2 points
...     print(clef) # Attention ne pas oublier la tabulation!!
```

Structures de données : Dictionnaires

21

```
>>> for clef in dico: # Ne pas oublier les 2 points
...     print (clef, dico[clef]) # Attention ne pas oublier la tabulation!!
>>> for clef, value in dico.items(): # Ne pas oublier les 2 points
...     print(clef, value) # Attention ne pas oublier la tabulation!!
>>> dico2={'ordinateur': 'computer', 'souris' : 'mouse'}
>>> print(dico2)
>>> dico2=dico # Attention dico et dico2 correspondent au même dictionnaire!
>>> print(dico)
>>> del dico2['computer']
>>> print(dico)
>>> print(dico2)
>>> dico3=dico.copy() # dico3 est une copie du dictionnaire reference par dico!
>>> del dico3['keyboard']
>>> print(dico)
>>> print(dico3)
>>> help(dict) # pour l'aide sur les dictionnaires
```

Références et adresses

22

En python, tout est objet, y compris les valeurs. Les variables sont des références à des valeurs.

Exemple 8 : Références et adresses

```
>>> help(id) # pour afficher la description de la fonction retournant une adresse
>>> id(2) # on affiche l'adresse de la valeur 2
10455072
>>> a=2
>>> id(a)
10455072 # la variable 'a' a la même adresse que celle de la valeur 2
>>> b=2
>>> id(b)
10455072 # la variable 'b' a la même adresse que celle de la valeur 2
>>> c=b
>>> id(c)
10455072 # la variable 'c' a la même adresse que celle de la valeur 2
>>> d=b
>>> id(d)
```

Références et adresses

23

```
10455072 # la variable 'd' a la même adresse que celle de 'b'
>>> d=3
>>> id(d) # l'adresse de 'd' a changee
10455104
>>> id(3)
10455104
>>> list=[1,2,3]
>>> id(list)
140482820775816
>>> list2=list
>>> id(list2)
140482820775816 # list et list2 ont la même adresse
>>> id([1,2,3])
140482820774664 # Adresse de la liste [1,2,3] différente de celle de list et list2
>>> list3=[1,2,3]
>>> id(list3)
140482820772552 # adresse différente de celle de list, list2 et [1,2,3]
>>> id(list[0])
10455040
>>> id(list3[0])
10455040 # adresse de list[0] et list3[0]
>>> id(1)
10455040 # adresse de la valeur 1
```

Références et adresses

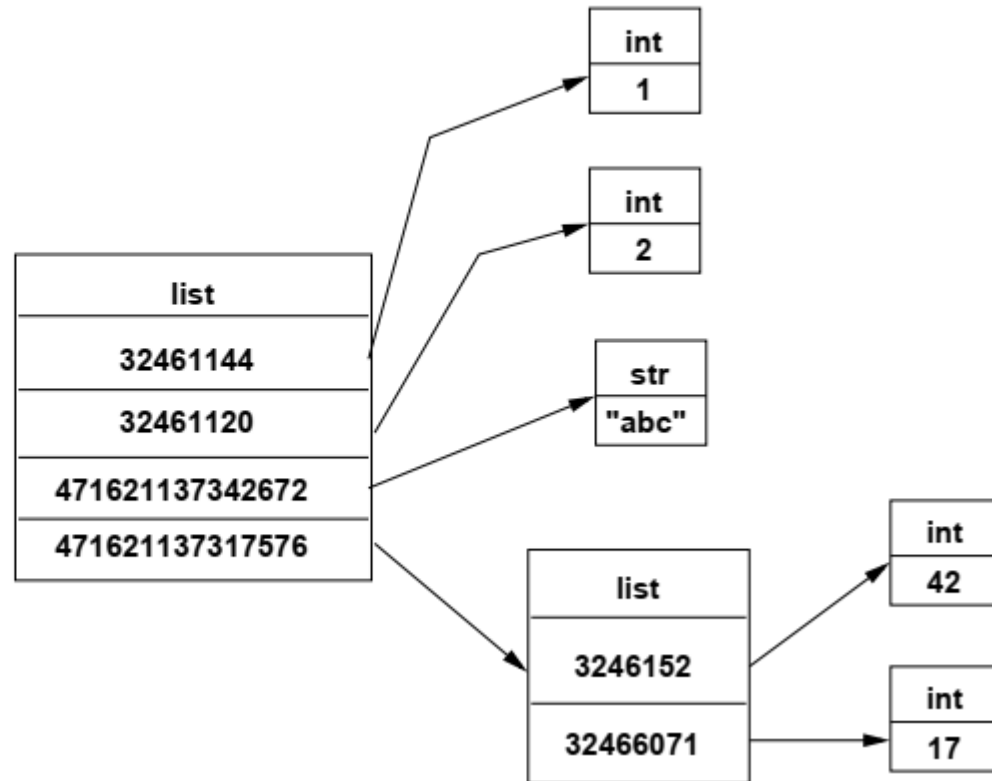
24

```
>>> list is list2 # Pour tester l'identite memoire
True
>>> list[0] is 1
True
>>> list is list3
False # list et list3 correspondent a la mÃªme liste
      # mais a 2 emplacement memoire differents
>>> list[0] is list3[0]
True
```

Références et adresses

25

Un exemple de représentation en mémoire d'une liste est donné dans la figure³ 9.



Fonctions: Fonction simple sans paramètres

26

Exercice 10 : Définir une fonction sans paramètre

Tapez le code suivant et observez le résultat.

```
>>> def function(): # Definition de fonction sans parametre - Ne pas oublier les :  
...     n=10 # Faire une tabulation en debut de ligne  
...     while n>0: # Faire une tab. en debut de ligne et ne pas oublier les :  
...         print(n/2, n%2) # Faire 2 tabulations en debut de ligne  
...         n=n-1 # Faire 2 tabulations en debut de ligne  
# Tapez encore une fois <Enter> si vous etes en ligne de commande  
>>>function() # Appel de la fonction
```

Exercice 11 : Définir une fonction sans paramètre qui appelle une autre fonction

Tapez le code suivant et observez le résultat.

```
>>> def function2(): # Ne pas oublier les : et une tab. sur la ligne suivante  
...     print('Affichage du resultat et du reste de la division des entiers de 10 \  
...     function() # Faire une tabulation en debut de ligne  
# Tapez encore une fois <Enter> si vous etes en ligne de commande  
>>> function2()
```

Fonctions: Fonction simple avec paramètres

27

Exercice 12 : Définir une fonction à 1 paramètre

Tapez le code suivant et observez le résultat.

```
def function(n): # Definition d'une fonction a un parametre
...     while n>0: # Faire une tab. en debut de ligne et ne pas oublier les :
...         print(n/2, n%2) # Faire 2 tabulations en debut de ligne
...         n=n-1 # Faire 2 tabulations en debut de ligne
# Tapez encore une fois <Enter> si vous etes en ligne de commande
>>> function(3) # Appel de la fonction avec le parametre 3
# Tapez encore une fois <Enter> si vous etes en ligne de commande
>>> function(5) # Appel de la fonction avec le parametre 5
```

Exercice 13 : Définir une fonction à 2 paramètres

Tapez le code suivant et observez le résultat.

```
>>> def function(entree,diviseur): # Definition d'une fct. a plusieurs parametres
...     while(entree>0):
...         print(entree/diviseur, entree%diviseur)
...         entree=entree-1
# Tapez encore une fois <Enter> si vous etes en ligne de commande
>>> function(10,2)
```

Fonctions: Fonction simple avec paramètres

28

Exercice 14 : Tapez le code suivant et observez le résultat.

```
>>> def afficher3fois(arg):  
...     print(arg, arg, arg)  
# Tapez encore une fois <Enter> si vous êtes en ligne de commande  
>>> afficher3fois(3)  
>>> afficher3fois('exemple')  
>>> afficher3fois([3,4])  
>>> afficher3fois(3*4)
```

Fonctions: valeur par défaut des paramètres

29

Exercice 16 : Tapez le code suivant et observez le résultat.

```
>>> def function(entree,diviseur=2): # fct. avec valeur par default pour diviseur
...     while(entree>0):
...         print(entree/diviseur, entree%diviseur)
...         entree=entree-1
# Tapez encore une fois <Enter> si vous Ãªtes en ligne de commande
>>> function(10)
```

Exercice 17 : Tapez le code suivant et observez le résultat.

```
>>> def function(entree=10,diviseur=2):
...     while(entree>0):
...         print(entree/diviseur, entree%diviseur)
...         entree=entree-1
# Tapez encore une fois <Enter> si vous Ãªtes en ligne de commande
>>> function()
>>> function(4,2)
>>> function(diviseur=4,entree=12)
```

Fonctions: Affecter une instance de fonction à une variable

30

Exercice 18 : Tapez le code suivant et observez le résultat.

```
>>>def multiplication(n,p):  
...     return n*p # pour retourner une valeur  
# Tapez encore une fois <Enter> si vous êtes en ligne de commande  
>>> a=multiplication(3,6)  
>>> a
```

Fonctions: Fonction avec un nombre variable de paramètres

31

Exemple 9 :

```
# les valeurs des arguments saisis vont etre stockes dans un t-uple - cf. print(type(  
def fonction_test(*args):  
    print("type de args :", type(args))  
    for arg in args:  
        print("paramètre:", arg)  
        print()  
  
fonction_test()  
fonction_test(1)  
fonction_test(1, "a")  
fonction_test(1, "a", 3)
```

Exercice 19 : *Ecrire une fonction qui calcule la somme des entiers passés en paramètres.*

Fonctions: Passage des paramètres : immuable et non immuable

32

En Python, il existe en effet deux types d'objets: les non immuables ou *mutables* tels que les listes, les dictionnaires, etc., que l'on peut modifier après leur création, et les immuables ou *non mutables* tels que les strings, int, floats, tuples, etc., que l'on ne peut pas modifier après leur création.

Exercice 20 : Tapez le code de cette mauvaise fonction *echange* dans un fichier, exécutez le et analysez le résultat.

```
def incr(a):  
    print("adresse a: ",id(a))  
    a=a+1  
    print("Dans la fonction a =",a)  
  
def ajouter(l):  
    l=l.append(5)
```

Fonctions: Passage des paramètres : immuable et non immuable

33

En Python, il existe en effet deux types d'objets: les non immuables ou *mutables* tels que les listes, les dictionnaires, etc., que l'on peut modifier après leur création, et les immuables ou *non mutables* tels que les strings, int, floats, tuples, etc., que l'on ne peut pas modifier après leur création.

Exercice 20 : Tapez le code de cette mauvaise fonction *echange* dans un fichier, exécutez le et analysez le résultat.

```
def incr(a):  
    print("adresse a: ",id(a))  
    a=a+1  
    print("Dans la fonction a =",a)  
  
def ajouter(l):  
    l=l.append(5)
```

Fonctions: Passage des paramètres : immuable et non immuable

34

```
a=3
print("adresse a: ",id(a))
incr(a)
print("Après l'appel a=",a)
```

```
l=[1,2,3]
ajouter(l)
print("l=: ",l)
```

Fonctions: Variable locale/variable globale

35

Exemple 10 : Mauvaise fonction echange

```
def echange(a,b):  
    print("adresses des parametres : ", id(a),id(b))  
    c=a  
    a=b  
    b=c
```

```
x,y = 2,3  
print("adresses de x et de y : ", id(x),id(y))  
echange(x,y)  
print("x=",x)  
print("y=",y)
```

Si vous exécutez le code précédent, vous verrez que les variables x et y n'ont pas été échangées car il s'agit de variables non immuables.

Fonctions: Variable locale/variable globale

36

Exemple 11 : Exemple de fonction **echange** qui fonctionne

```
def echange2(a,b):  
    c=a  
    a=b  
    b=c  
    return a,b
```

```
x,y = 2,3  
x,y=echange2(x,y) # les valeurs de x et y sont bien echangees car  
                  # on les modifie leur valeur ici  
  
print("x=",x)  
print("y=",y)  
print() # pour sauter une ligne  
  
a=2  
b=3
```

Fonctions: Variable locale/variable globale

(37)

```
a,b=echange2(a,b) # les valeurs de a et b sont bien echangees car
                  # on les modifie leur valeur ici

print("a=",a)
print("b=",b)
print()
```

Exemple 12 : Autre exemple de fonction echange qui fonctionne

```
def echangeab():
    global a # a est la variable globale du programme
    global b # b est la variable globale du programme
    c=a
    a=b
    b=c

a=2
b=3
echangeab()
print("a=",a)
print("b=",b)
```

Fonctions: Fonction anonyme (lambda function)

38

Exercice 21 : Tapez le code⁴ suivant et observez le résultat.

```
>>> def f (x): return x**2
>>> print(f(8))
>>> g = lambda x: x**2
>>> print(g(8))
>>> (lambda x: x*2)(3) //2
>>> def make_incrementor (n): return lambda x: x + n
>>> f = make_incrementor(2)
>>> g = make_incrementor(6)
>>> print(f(42), g(42))
>>> print(make_incrementor(22)(33))
```

Fonctions: Fonction anonyme (lambda function)

39

Exercice 21 : Tapez le code⁴ suivant et observez le résultat.

```
>>> def f (x): return x**2
>>> print(f(8))
>>> g = lambda x: x**2
>>> print(g(8))
>>> (lambda x: x*2)(3) //2
>>> def make_incrementor (n): return lambda x: x + n
>>> f = make_incrementor(2)
>>> g = make_incrementor(6)
>>> print(f(42), g(42))
>>> print(make_incrementor(22)(33))
```

Fichiers: Instanciation du répertoire courant

40

Exemple 13 : *modifier le répertoire courant de l'interpréteur*

```
>>>from os import chdir
>>>chdir("/home/login/exercices") # Mettre ici le repertoire
                                   # de stockage de vos exercices
```

La première ligne permet d'importer la commande `chdir()` du module `os` contenant une série de fonctions permettant de dialoguer avec le système d'exploitation (`os` = operating system).

La deuxième ligne permet de spécifier le répertoire courant de l'interpréteur python, i.e. le répertoire où il va récupérer les fichiers.

Fichiers: Manipulation de fichiers

41

Exercice 22 : Tapez le code suivant et observez le résultat.

```
>>> f=open('test.txt','w') # Pour ouvrir un fichier en mode ecriture
>>> f.write("Bonjour\n") # Pour ecrire dans le fichier
>>> f.write("Ceci est un test d'ecriture dans un fichier")
>>> f.close() # Pour fermer le fichier
           " verifier le fichier dans un editeur de texte
>>> f=open('test.txt','r') # # Pour ouvrir un fichier en mode lecture
>>> b=f.read() # Pour lire tout le fichier
>>> print(b)
>>> f.close() # Pour fermer le fichier
>>> f=open('test.txt','r') # # Pour ouvrir un fichier en mode lecture
>>> b=f.read(3) # Pour lire 3 caracteres
               # a partir de la position courante du curseur
```

Fichiers: Manipulation de fichiers

42

```
>>> print(b)
>>> f.close()
>>> f=open('test.txt','r')
>>> b = f.readline() # Pour lire une ligne dans le fichier
>>> print(b)
>>> b = f.readlines() # Pour lire toutes les lignes et les stocker dans une liste
>>> print(b)
>>> f.close()
>>> f=open('test.txt','a') # Pour ouvrir le fichier en mode ajout
>>> f.write("\n Fin")
>>> f.close()
>>> f=open('test.txt','r')
>>> b = f.readlines()
>>> print(len(b))
>>> print(b)
>>> f.close()
>>> f=open('test.txt','r')
>>> print(f.read()) # Pour lire tout le fichier
                        # deplace le curseur à la fin du fichier
>>> print(f.read())
>>> f.close()
```

Fichiers: Copier des variables dans un fichier

43

Exercice 24 : Tapez le code suivant⁵ et observez le résultat.

```
>>> a = 5
>>> b = 2.83
>>> c = 67
>>> f = open('test.txt', 'w')
>>> f.write(str(a)) # Pour ecrire un entier converti en string
>>> f.write(str(b))
>>> f.write(str(c))
>>> f.close()
>>> f = open('test.txt', 'r')
>>> print(f.read())
>>> f.close()
```

Fichiers: Copier des variables dans un fichier

44

```
>>> import pickle # Pour importer un module permettant
                    # de conserver le type des variables
>>> f = open('Monfichier', 'wb') # ouverture d'un fichier binaire
>>> pickle.dump(a, f) # Pour enregistrer dans le fichier
>>> pickle.dump(b, f) # equivalent a write mais en conservant le type de
>>> pickle.dump(c, f)
>>> f.close()
>>> f = open('Monfichier', 'rb')
>>> print(f.read())
>>> f.close()
>>> f = open('Monfichier', 'rb')
>>> t = pickle.load(f) # Operation inverse de dump
>>> print(t, type(t))
>>> t = pickle.load(f) # Equivalent a read mais en conservant le type de
>>> print(t, type(t))
>>> t = pickle.load(f)
>>> print(t, type(t))
>>> f.close()
```

Exercices

45

1. Écrire une boucle de programme qui demande à l'utilisateur d'entrer des notes d'élèves. La boucle se terminera seulement si l'utilisateur entre une valeur négative. Avec les notes ainsi entrées, construire progressivement une liste. Après chaque entrée d'une nouvelle note (et donc à chaque itération de la boucle), afficher le nombre de notes entrées, la note la plus élevée, la note la plus basse, la moyenne de toutes les notes.

Exercices

46

- 1) Définissez une fonction `maximum(n1,n2,n3)` qui renvoie le plus grand de 3 nombres `n1`, `n2`, `n3` fournis en arguments. Par exemple, l'exécution de l'instruction :

`print(maximum(2,5,4))` doit donner le résultat : 5.

- 1) Définissez une fonction `indexMax(liste)` qui renvoie l'index de l'élément ayant la valeur la plus élevée dans la liste transmise en argument.

Exemple d'utilisation : `serie = [5, 8, 2, 1, 9, 3, 6, 7]`

`print(indexMax(serie))`

4

- 1) Définissez une fonction `nomMois(n)` qui renvoie le nom du *énième* mois de l'année. Par exemple, l'exécution de l'instruction :

`print(nomMois(4))` doit donner le résultat : Avril.

Exercices

47

- 1) Définissez une fonction `changeCar(ch,ca1,ca2,debut,fin)` qui remplace tous les caractères `ca1` par des caractères `ca2` dans la chaîne de caractères `ch`, à partir de l'indice `debut` et jusqu'à l'indice `fin`, ces deux derniers arguments pouvant être omis (et dans ce cas la chaîne est traitée d'une extrémité à l'autre). Exemples de la fonctionnalité attendue :
- ```
>>> phrase = 'Ceci est une toute petite phrase.'
>>> print(changeCar(phrase, ' ', '*')) Ceci*est*une*toute*petite*phrase.
>>> print(changeCar(phrase, ' ', '*', 8, 12)) Ceci est*une*toute petite phrase.
>>> print(changeCar(phrase, ' ', '*', 12)) Ceci est une*toute*petite*phrase.
>>> print(changeCar(phrase, ' ', '*', fin = 12)) Ceci*est*une*toute petite phrase.
```

# Exercices

48

- 1) Considérons que vous avez à votre disposition un fichier texte contenant des phrases de différentes longueurs. Écrivez un script qui recherche et affiche la phrase la plus longue.
- 2) Écrivez un script qui génère automatiquement un fichier texte contenant les tables de multiplication de 2 à 30 (chacune d'entre elles incluant 20 termes seulement).
- 3) Écrivez un script qui recopie un fichier texte en triplant tous les espaces entre les mots.